Signals and Systems

Computer Assignment 1

Report

Parsa Taherkhani

810102579

Part 1 - MATLAB Familiarization

No report or heavy implementation is needed here

Part 2 - Linear System Identification

```
%% Part 2 - Linear System Identification
clc; clear; close all;

% Load the given data
load('p2.mat');
```

Given linear system:

$$y(t) = \alpha x(t) + \beta$$

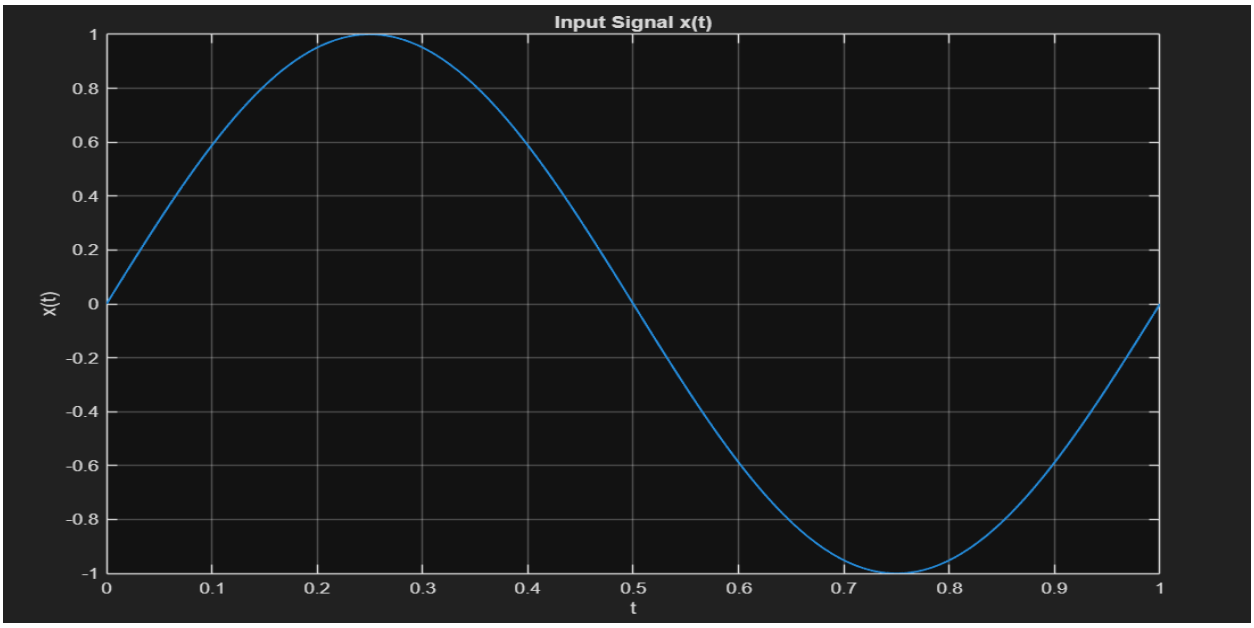
Where:

- $x(t)$ is the input signal
- $y(t)$ is the output signal
- α and β are unknown constants to be estimated

2-1

Load and plot input x versus t

```
%% 2-1 Plot x(t)
figure;
plot(t, x, 'LineWidth', 1.2);
xlabel('t');
ylabel('x(t)');
title('Input Signal x(t)');
grid on;
```



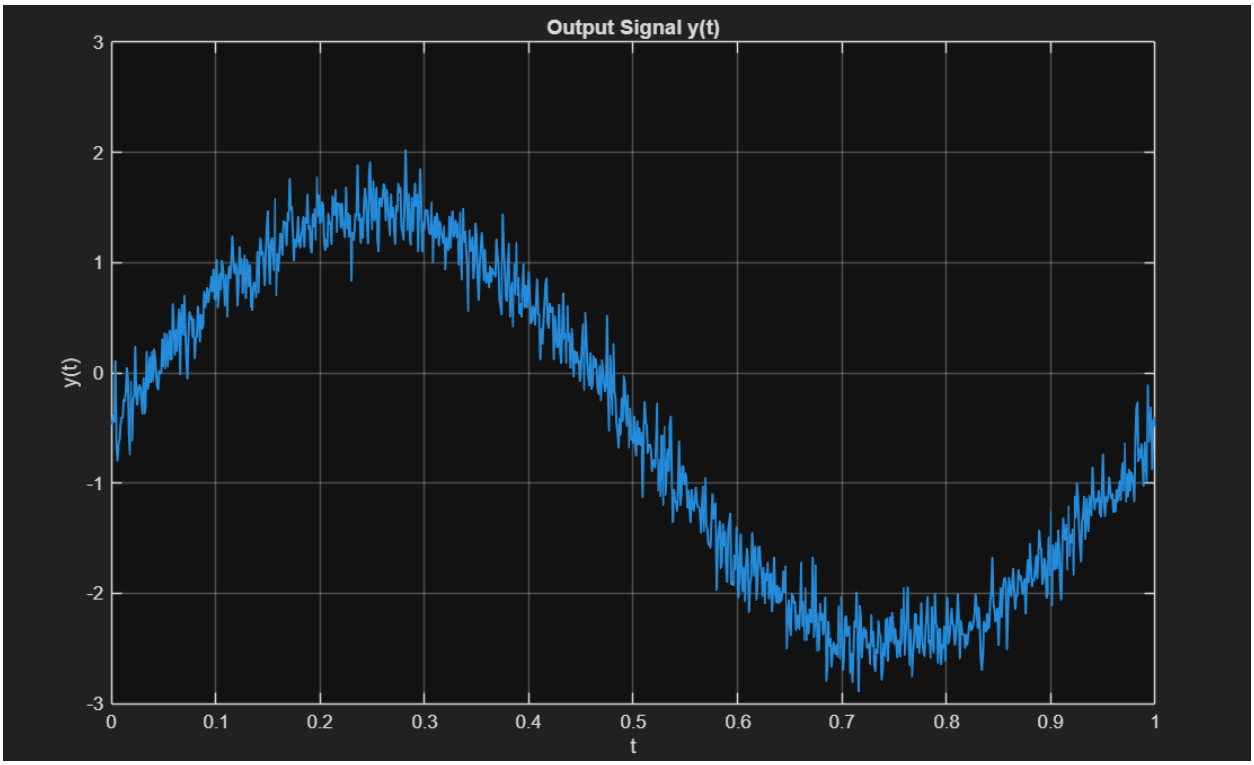
2-2

Load and plot output y versus t

- Code

```
%% 2-2 Plot y(t)
figure;
plot(t, y, 'LineWidth', 1.2);
xlabel('t');
ylabel('y(t)');
title('Output Signal y(t)');
grid on;
```

- Plot



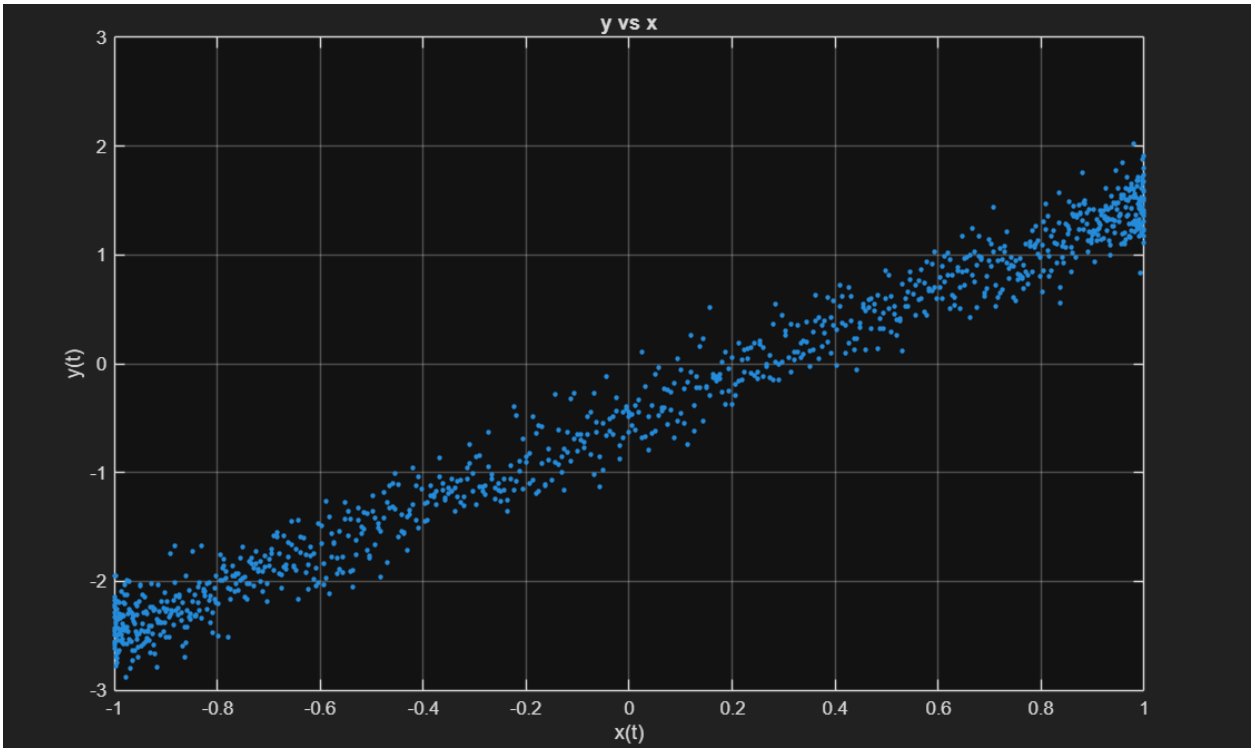
2-3

scatter plot y vs x to visualize the linear relation

- Code:

```
%% 2-3 Scatter Plot y vs x
figure;
plot(x, y, '.', 'MarkerSize', 8);
xlabel('x(t)');
ylabel('y(t)');
title('y vs x');
grid on;
```

- Plot:



You can see that the points roughly form a line which confirms linearity.

2-4

We must minimize the cost function

$$f(\alpha\ \beta) = \sum_{t}(y(t) - \alpha x(t) - \beta)\text{^}2$$

This is equivalent to linear regression.
Analytical least-squares solution:

$$[\alpha\ \beta] = (X^T X)^{-1} X^T y$$

where $X = [x\ 1]$

- Implementation :

p2_4 estimates the parameters alpha and beta in $y(t) = \alpha x(t) + \beta$ using least squares minimization.

It is a robust and efficient approach to determine system parameters (α, β) from experimental data, even in the presence of measurement noise.
This method is widely used in system identification, signal processing, and data fitting applications.

This function implements a standard **linear regression** approach to find the best-fitting straight line that relates the input signal $x(t)$ and the output signal $y(t)$.

```
function [alpha, beta] = p2_4(x, y)
    % Ensure column vectors
    x = x(:);
    y = y(:);

    % Construct design matrix
    X = [x ones(size(x))];

    % Solve for [alpha; beta]
    params = (X' * X) \ (X' * y);

    alpha = params(1);
    beta  = params(2);
end
```

To represent the linear model $y = \alpha x + \beta$ in matrix form, we rewrite it as:

$$\boldsymbol{y} = \boldsymbol{X\theta}$$

where:

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x_1} & \mathbf{1} \\ \vdots & \vdots \\ \boldsymbol{x_N} & \mathbf{1} \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} \boldsymbol{\alpha} \\ \boldsymbol{\beta} \end{bmatrix}$$

and $\mathbf{y} = [\boldsymbol{y_1}, \boldsymbol{y_2}, \dots, \boldsymbol{y_N}]^{\boldsymbol{T}}$

Interpretation of Results

- The slope α represents the gain of the system — how much the output $y$changes for a unit change in the input x.
- The intercept β represents a constant bias or offset added to the output.

Thus, this function provides a numerical estimation of the linear relationship between the input and output data of a given system.

- Testing p2_4

```
%% Test p2_4 function
alpha_true = 2.5;
beta_true  = -1.2;
t = 0:0.001:1;
x = sin(2*pi*5*t);     % sample input
y_clean = alpha_true*x + beta_true;

% Add small Gaussian noise
noise = 0.05*randn(size(y_clean));
y_noisy = y_clean + noise;

% Estimate parameters from noisy data
[alpha_est, beta_est] = p2_4(x, y_noisy);

fprintf('True alpha = %.3f, Estimated alpha = %.3f\n', alpha_true, alpha_est);
fprintf('True beta  = %.3f, Estimated beta  = %.3f\n', beta_true, beta_est);
```

Result

```
>> test_p2_4
True alpha = 2.500, Estimated alpha = 2.499
True beta  = -1.200, Estimated beta  = -1.199
```

Mathematical background:

$$f(\alpha,\beta) = \sum_t (y(t) - \alpha x(t) - \beta)^2$$

$$= \sum_t [y^2(t) + \alpha^2 x^2(t) + \beta^2 - 2\alpha x(t)y(t) - 2\beta y(t) + 2\alpha\beta x(t)]$$

$$\xrightarrow{for\ minimizing\ f(\alpha,\beta)} \frac{\partial f}{\partial \alpha} = 0\ and\ \frac{\partial f}{\partial \beta} = 0$$

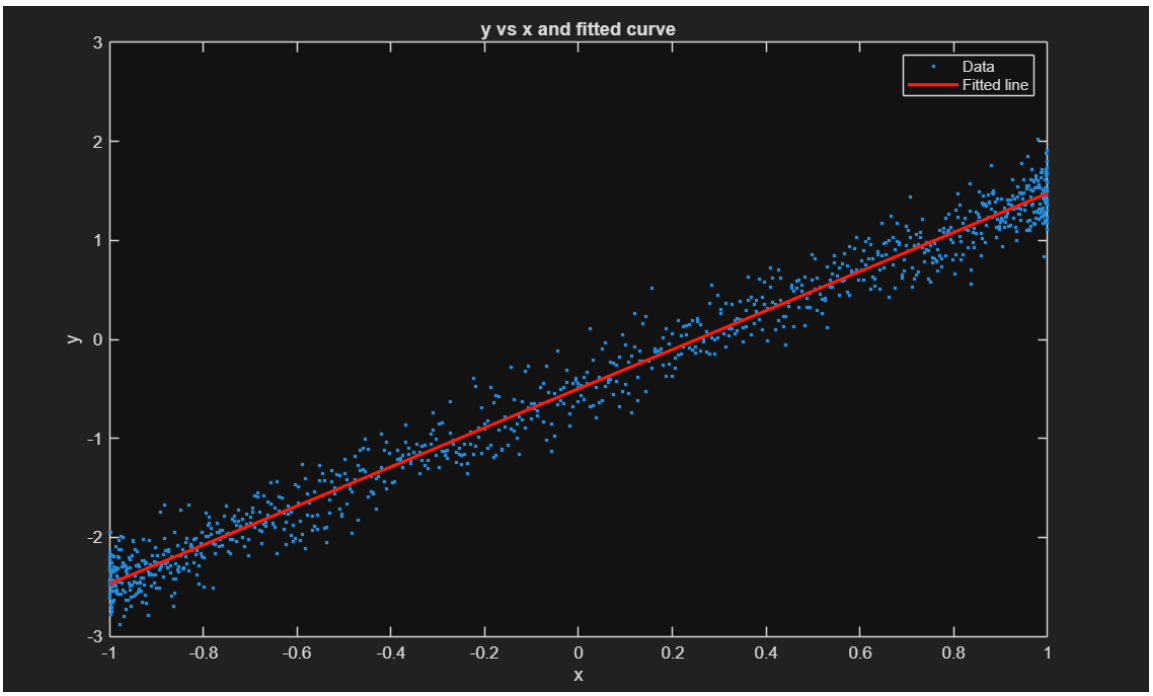
$$\frac{\partial f}{\partial \alpha} = 0 \xrightarrow{yields} \sum_t 2\alpha x^2(t) - 2x(t)y(t) + 2\beta x(t) = 0 \xrightarrow{yields} \alpha \sum_t x^2(t) + \beta \sum_t x(t)$$

$$= \sum_t x(t)y(t)$$

$$\frac{\partial f}{\partial \beta} = 0 \xrightarrow{yields} \sum_t 2\beta - 2y(t) + 2\alpha x(t) = 0 \xrightarrow{yields} \alpha \sum_t x(t) + \beta \sum_t 1 = \sum_t y(t)$$

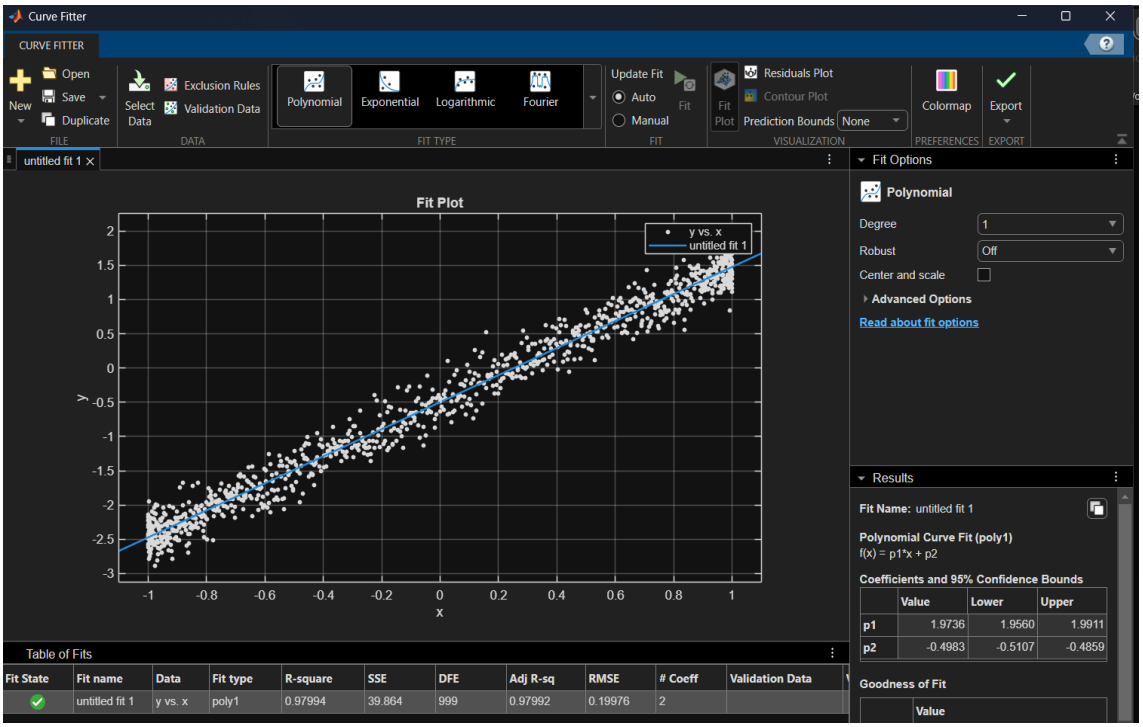
$$\begin{bmatrix} \sum_t x^2(t) & \sum_t x(t) \\ \sum_t x(t) & \sum_t 1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum_t x(t)y(t) \\ \sum_t y(t) \end{bmatrix} \xrightarrow{yields} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum_t x^2(t) & \sum_t x(t) \\ \sum_t x(t) & \sum_t 1 \end{bmatrix}^{-1} \begin{bmatrix} \sum_t x(t)y(t) \\ \sum_t y(t) \end{bmatrix}$$

- Plot of Fitted Line




```
Estimated alpha = 1.974
Estimated beta  = -0.498
```

2-5



as you can see the results for α (slope) and β (intercept) are exactly same for both Curve Fitter and p2_4 function that I have implemented.

Part 3 - Radar Distance Estimation

Objective

The goal of this experiment is to estimate the distance between a radar and a reflecting object using template matching (correlation).
By transmitting a known pulse signal and receiving its delayed echo, the round-trip delay is measured and used to calculate the target distance.

Theoretical Background

When a radar transmits a pulse $x(t)$, the received signal after reflection is:

$$y(t) = \alpha x(t - t_d)$$

where:

- α : reflection coefficient (attenuation)
- $t_d = \frac{2R}{c}$: round-trip delay
- R : target distance
- $c = 3 \times 10^8$ m/s: speed of light

The **correlation function** between the transmitted and received signals is given by:

$$r_{xy}(\tau) = \int x(t)y(t + \tau)dt$$

The value of τthat **maximizes** this correlation corresponds to the **time delay** t_d.
Once t_d is found, the estimated distance is:

$$R = \frac{ct_d}{2}$$

```
%% Parameters
c   = 3e8;
ts  = 1e-9;
T   = 1e-5;
tau = 1e-6;
t = 0:ts:T;

alpha = 0.5;
R_true = 450;
td = 2 * R_true / c;
index = round(tau / ts);
```

Implementation Steps

3-1 Transmitted Signal

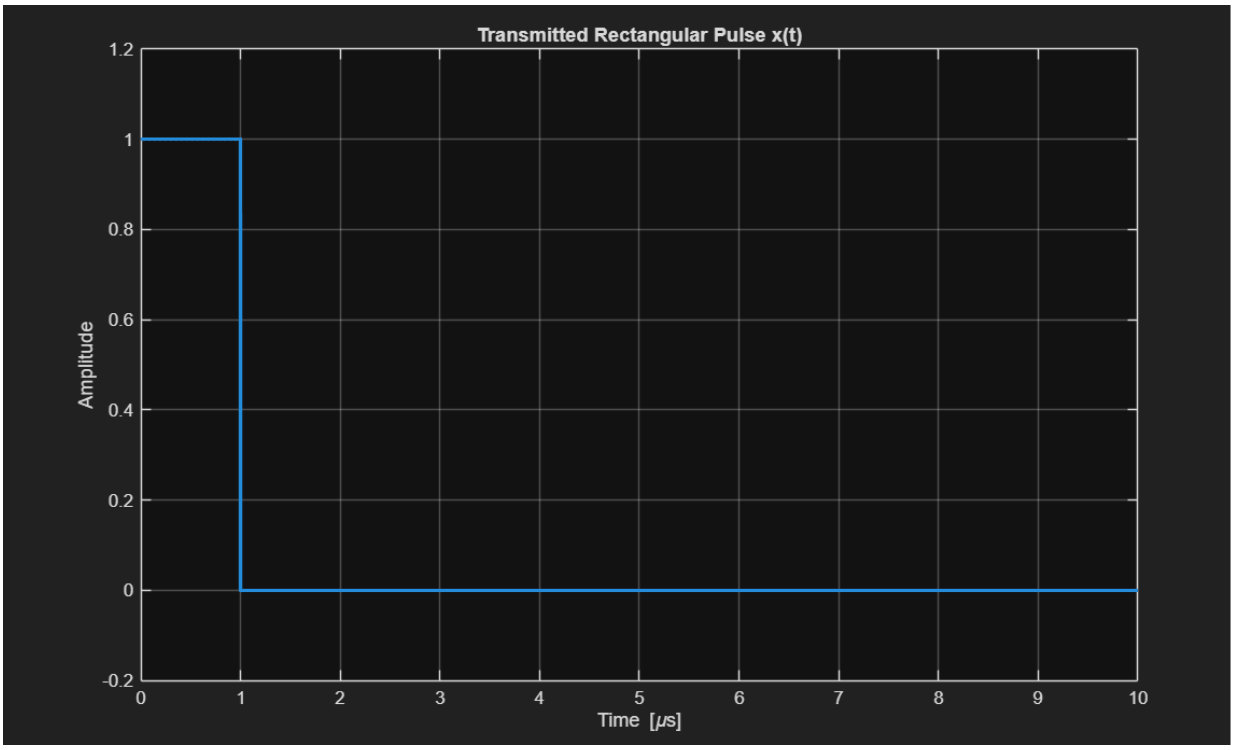
A rectangular pulse of duration $\tau = 1\ \mu s$ is generated with a sampling period $t_s = 1\ ns$. This signal is simple and ensures a sharp, unique correlation peak.

- Code

```
%% 3-1: Transmitted Signal
x = zeros(size(t));
x(1:index) = 1;

figure;
plot(t * 1e6, x, 'LineWidth', 2);
xlabel('Time [\mus]');
ylabel('Amplitude');
title('Transmitted Rectangular Pulse x(t)');
xlim([0 T * 1e6]);
ylim([-0.2 1.2]);
grid on;
```

- Plot



3-2 Received Signal

The received signal is a delayed and attenuated version of the transmitted pulse:

$$y(t) = 0.5x(t - t_d)$$

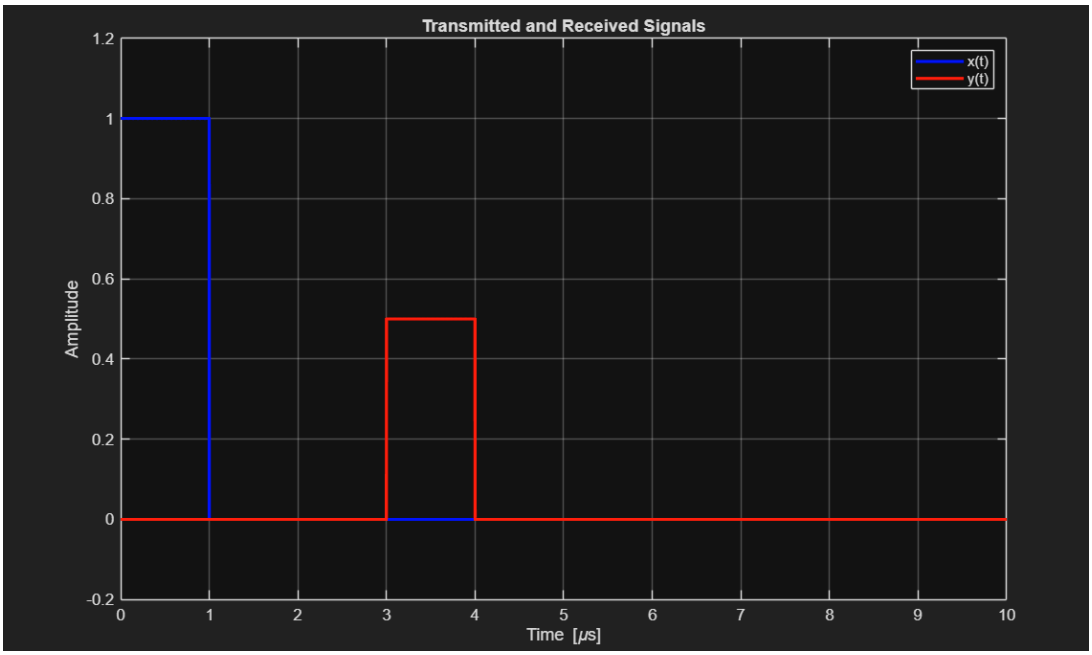
where $t_d = \frac{2R}{c} = 3\ \mu s$ for a target at $R = 450\ m$.

- Code

```
%% 3-2 Generate Recieved Signal
c=3e8;
R=450;
td=3e-6;
startIndex=round(td/ts);
recievedSignal=zeros(size(t));
recievedSignal(startIndex+1:startIndex+index)=0.5;
hold on;
figure;
plot(t,recievedSignal,'r');
ylim([-1 1]);
xlim([0 T]);
```

$$ct_d = 2R \xrightarrow{yields} t_d = \frac{2R}{c} = \frac{2 * 450}{3 * 10^8} = 3 * 10^{-6} s$$

- Plot



3-3 Correlation Calculation

A template matching process slides a replica of the transmitted pulse across the received signal.

At each time shift, the correlation is computed as:

$$r(i) = \sum_{t} x(t - i) \cdot y(t)$$

The delay t_d is determined from the position of the maximum correlation value.
The corresponding distance is computed as $R = \frac{c\, t_d}{2}$.

- Code

```
%% 3-3: Correlation Function
corrVals = zeros(size(t));
finalIndex = length(t);

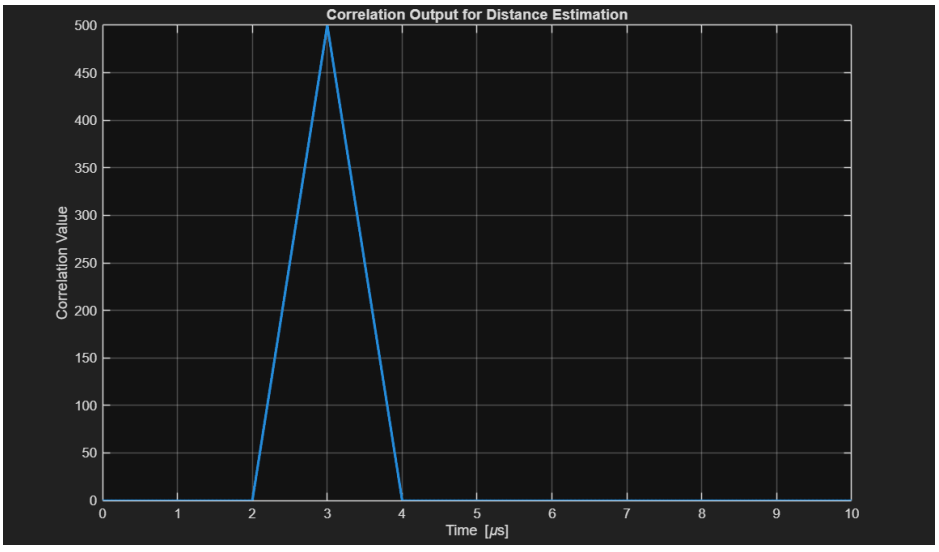
for i = 1:finalIndex - index
    shiftedPulse = zeros(size(t));
    shiftedPulse(i:i+index-1) = 1;
    corrVals(i) = sum(shiftedPulse .* y);
end

[~, maxIndex] = max(corrVals);
td_est = t(maxIndex);
R_est = (td_est * c) / 2;

fprintf('True R = %.2f m | Estimated R = %.2f m\n', R_true, R_est);

figure;
plot(t * 1e6, corrVals, 'LineWidth', 2);
xlabel('Time [\mus]'); ylabel('Correlation Value');
title('Correlation Output for Distance Estimation');
grid on;
```

- Plot



3-4 Effect of Noise on Distance Estimation

In the radar model used in this experiment, the received signal is a delayed and attenuated version of the transmitted pulse:

$$y(t) = \alpha x(t - t_d) + n(t)$$

As the power (variance) of noise increases, the correlation peak becomes less distinct, making it harder to detect the correct delay and therefore increasing the estimation error.

Implementation

In this section, Gaussian noise was added to the received signal with different standard deviation values ranging from 0 to 5.
For each noise level:

1. A noisy received signal was generated using:

$$y_{\text{noisy}}(t) = y(t) + \text{Noise}(t)$$

where the noise is created using MATLAB's randn() function.

2. The distance was estimated using the same correlation method developed in Part 3-3.
3. The experiment was repeated **100 times** for each noise level to ensure statistical reliability.
4. The **average estimation error** was computed as:

$$\text{Error} = \frac{1}{N}\sum_{i=1}^{N} \quad | R_{\text{est}} - R_{\text{true}} |$$

Finally, the average error for each noise power was plotted against the corresponding noise standard deviation.

```
%% 3-4: Effect of Noise
noiseLevels = 0:0.1:5;
numIterations = 100;
estimationError = zeros(size(noiseLevels));

for i = 1:length(noiseLevels)
    noiseStd = noiseLevels(i);
    totalError = 0;

    for j = 1:numIterations
        noisySignal = y + noiseStd * randn(size(y));
        corrValsNoisy = zeros(size(t));

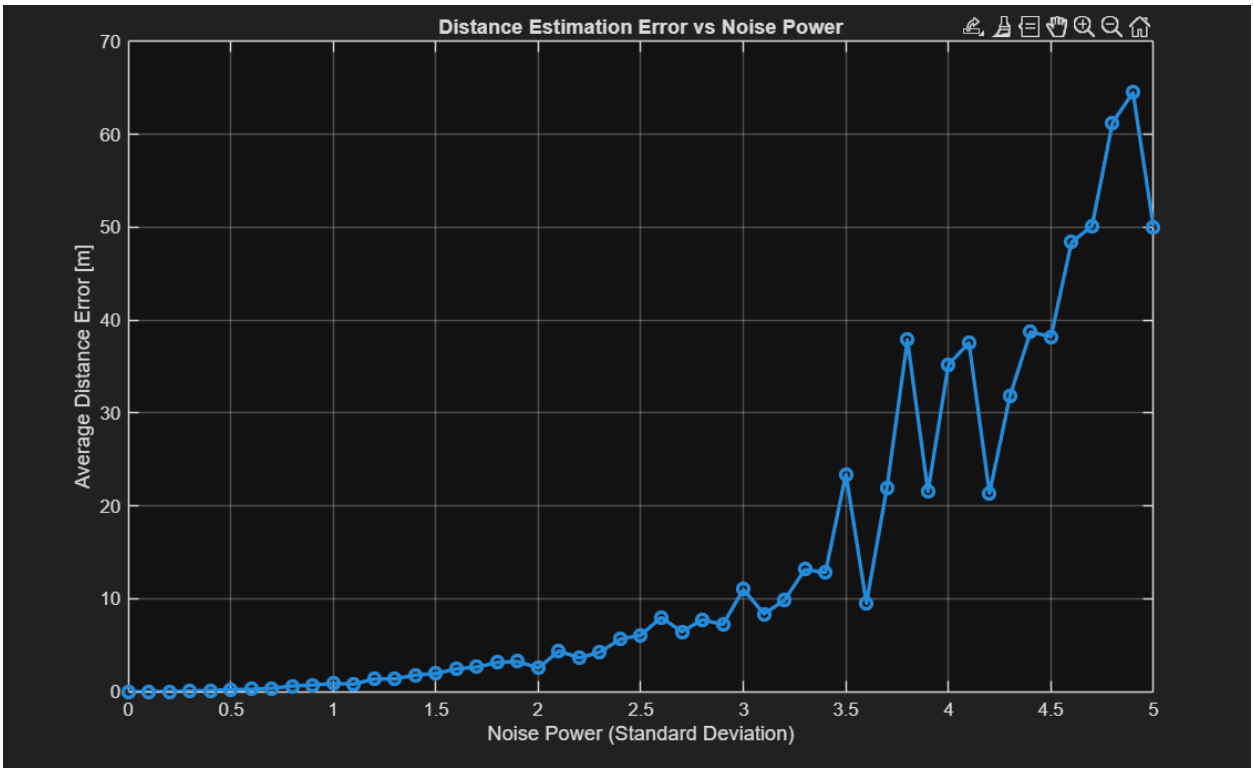
        for k = 1:(finalIndex - index + 1)
            shiftedPulse = zeros(size(t));
            shiftedPulse(k:k+index-1) = 1;
            corrValsNoisy(k) = sum(shiftedPulse .* noisySignal);
        end

        [~, maxIdx] = max(corrValsNoisy);
        td_noisy = t(maxIdx);
        R_noisy = (td_noisy * c) / 2;
        totalError = totalError + abs(R_noisy - R_true);
    end

    estimationError(i) = totalError / numIterations;
    fprintf('Noise %.1f -> Mean Error = %.2f m\n', noiseStd, estimationError(i));
end
```

```
figure;
plot(noiseLevels, estimationError, '-o', 'LineWidth', 2);
xlabel('Noise Power (Standard Deviation)');
ylabel('Average Distance Error [m]');
title('Distance Estimation Error vs Noise Power');
grid on;

disp('Simulation completed successfully.');
```



Results and Discussion

The plot above shows the relationship between **noise power (expressed as standard deviation)** and the **average distance estimation error** in meters. Each data point represents the mean of 100 Monte Carlo simulations for a specific noise level.

Trend Analysis

- For **low noise levels** (standard deviation below ≈ 1), the average estimation error is extremely small—typically less than 2 m.
  This indicates that the correlation method accurately detects the correct time delay when the received signal has a high signal-to-noise ratio (SNR).
  The correlation peak in this range remains sharp and well-defined.
- As the **noise power increases beyond σ ≈ 2**, a clear upward trend appears.
  The error grows gradually, showing that the correlation peak begins to distort and shift because of noise interference.
- Between **σ ≈ 3 and 5**, the estimation error rises sharply and becomes more erratic.
  In this region, the correlation function often exhibits **multiple competing peaks** or a **broadened maximum**, causing the algorithm to select incorrect delay values.
  The fluctuations visible in the plot are due to these random mis-detections from trial to trial.

Quantitative Behavior

- At moderate noise (σ ≈ 2.5–3.0), the mean distance error is roughly **10–15 m**, which corresponds to an uncertainty of about **30–50 ns** in time delay detection.
- At high noise (σ ≈ 4–5), the error increases rapidly to **50–65 m**, confirming the loss of reliable delay estimation when the SNR is very low.

Interpretation

This behavior aligns with theoretical expectations:

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}$$

As P_{noise} increases, the effective SNR decreases, reducing the distinguishability of the correlation peak.
When noise dominates the signal, the peak position becomes random, leading to large distance estimation errors.

Conclusion

The results demonstrate that:

1. The correlation-based delay estimation method performs **accurately and reliably** in low-noise environments.
2. As noise power increases, **estimation uncertainty grows non-linearly**, and the method becomes unreliable beyond a certain noise threshold.
3. Maintaining a high signal-to-noise ratio is therefore critical for precise radar distance measurement.

To mitigate this degradation in practice, engineers employ **matched filtering**, **averaging**, or **longer pulse durations** to improve detection robustness under noisy conditions.